Project Tracks

Detailed Track Descriptions for Capstone Project

Version 4. Everything that changed since version 3 is highlighted in yellow, like this. All of it is in Track 2.

🏞️ Project Tracks (Choose One or Propose Your Own)

Tracks 1–2 are set in **quick-response apparel manufacturing** and come with a small, clean dataset that you can read end-to-end in five minutes. **The data is deliberately not the challenge.** The challenge is building an agent with well-designed tools, an interface a real manager would use, and evaluation you can defend.

Tracks 3–4 are open-ended build tracks: you define the problem sharply, build a GenAI system, and prove it works.

In **every track, the user experience is graded**. Whoever your user is — a factory manager, a dispatcher, a researcher, a student — the interface they touch is part of the deliverable, not an afterthought bolted onto the model.

The setting for Tracks 1 and 2: quick-response apparel

Fast fashion changed how clothes get made. A brand no longer places one order for 100,000 units nine months ahead — it places a **small first drop**, watches what sells for a week, and then reorders the winners in **7 to 14 days**. For the factory that means hundreds of small orders in the building at once, reorders that arrive without warning, and peaks that are absorbed by renting capacity from outside workshops.

So the two hardest questions in the building are *"what is actually happening right now?"* and *"who should make this?"* Those are Tracks 1 and 2.

Read `Factory_Primer.pdf` first — three minutes, no manufacturing background needed. It is in your track folder.

Where your files are

Each track is **self-contained in its own folder**: take `track1/` or `track2/` and you have everything you need — the primer, the data, a half-page data dictionary, and (for Track 2) the simulator. Shared files are duplicated into both folders on purpose; you never need to look outside your own.

File	Rows	`track1/`	`track2/`	What it is
`Factory_Primer.pdf`	—	✓	✓	The three-minute domain primer
`data/orders.csv`	120	✓	✓	One customer order per row: who, what, how many, due when, current stage
`data/production_log.csv`	360	✓	—	Daily output per stage for 90 days
`data/workshops.csv`	8	✓	✓	A profile card per outside workshop: what it makes, capacity, current queue, defect rate, price, status
`data/dispatch_requests.txt`	30	—	✓	Track 2's inbox: allocation requests exactly as typed in the group chat

`data/data_dictionary.md`	—	✓	✓	Every column explained, in half a page
`harness/simulate.py`	—	—	✓	The shared Track 2 simulator with three baseline allocators
`data/orders_stages.csv`, `data/workshops_stages.csv`	120 / 8	—	✓	**Optional** stage extension for Track 2 (`simulate.py --stages`) — ignore unless you opt in
`CHANGELOG.md`	—	—	✓	What changed in the Track 2 folder, and what you need to re-run

No missing values and no traps in the CSVs; the only join is `order_id`. There are no usage restrictions — commit it, publish it, extend it. If your project needs a bigger playground you may extend the data; document what you changed.

Track 1: The General Manager's Co-Pilot

The problem: The general manager runs the factory on the move — office, floor, meetings. They do not write SQL, and their questions do not stop at 7 a.m.:

> *7:00 — "Did anything go wrong yesterday?" 9:40 — "How is the TrendCart order doing?" 11:15 — "A customer wants 800 hoodies by the 25th. Can we take it?" 14:30 — "That order hasn't moved in a week. Chase it up." 17:00 — "Remind me tomorrow if packing is still behind."*

Today each of these costs a phone call to a planner, a wait, and an answer that is sometimes wrong. **Goal**: build the co-pilot that handles them directly — a real assistant the manager talks to all day, not a dashboard and not a chat toy.

What the co-pilot does

1. **The morning briefing** — generated on a schedule, structured and prose: what happened yesterday, what is at risk, what needs a decision today.
2. **Answers anything, any time** — multi-turn, with fresh retrieval each turn. Concise first: the first reply fits in a glance — a few lines, the detail one follow-up away. Build it for whatever device you like; a laptop browser app is perfectly fine. (Voice in and voice out is a genuine plus — the manager is usually walking the floor.)
3. **Watches and alerts** — "*tell me if ORD-058 hasn't moved by Thursday*" becomes a standing check that fires on its own, not a note the manager must remember to ask about.
4. **Feasibility checks** — "*can we take 800 hoodies by the 25th?*" needs current load, typical stage rates, and workshop capacity. The answer is an estimate with stated assumptions, not a yes.
5. **Gets things done** — drafts the chase-up message, adds a note to an order, sets the reminder. **Anything with a side effect is confirmed before it happens** and logged after.

The centre of this track is the tool set, not the chat window

An LLM with a text box is not the deliverable. The deliverable is a system whose **tools** are designed for this job, each one specified before it is built. For every tool, your documentation must state: **what it takes in, what it returns, what question it exists to answer, and what it deliberately does not do.**

Five kinds of tool are the minimum. What is inside them is your design problem:

1. Retrieval — get the facts Query orders and daily output. Does the agent get one `run_sql` tool and full schema access, or a handful of narrow typed tools like `get_orders_at_risk(due_within_days)`? The first is flexible and hallucinates joins; the second is reliable and blind to anything you did not anticipate. **Pick one, and justify it against the failure mode you chose to accept.**

2. Judgement — decide what "normal" is "*Is 800 pieces from assembly normal?*" has no answer in any database. Someone has to define it: compared with last week? The same weekday? (The factory is closed on Sundays — a naive average is already wrong.) This must be a **tool with an explicit, inspectable definition** — not a sentence buried in a prompt where nobody can audit it.

3. Tracing — answer "why?" Every number the co-pilot states has to expand into the rows behind it. When the manager asks "*why is that order at risk?*", the agent must go back to the data rather than paraphrase what it already said. **A fabricated number is the worst failure in this track and is marked as such.**

4. Discovery — find what nobody asked about A briefing that only fills a fixed template misses the thing that matters, because the thing that matters is by definition unusual. What does a discovery tool look for — a stage whose output dropped, a customer whose orders are all slipping at once, an order that has gone quiet? How does it rank findings so that five items make the page and forty do not?

5. Action — make something happen The chase-up message, the reminder, the note on the order. Every action tool needs a confirmation step, an audit trail, and a stated blast radius: what is the worst thing this tool can do, and what stops it?

Design more tools than these five if the job needs them. A team that ships eight well-specified tools and a page of honest reasoning about the boundaries between them will outscore a team with a bigger model and no design.

What makes this hard (it is not the data)

- **The manager asks vague questions.** "*How is the TrendCart order doing?*" — TrendCart has a dozen orders. Ask, do not guess.
- **Some questions have no answer in this data.** There are no prices, no revenue, no worker names. An agent that says "*I cannot answer that — this system holds no price data*" scores higher than one that invents a plausible figure.
- **The LLM must not do the arithmetic.** Totals, averages and comparisons come from tools; the model narrates. This is the single most common source of wrong answers.
- **Proactive is a dial, not a switch.** A co-pilot that pings ten times a day gets muted. What earns an interruption, and what waits for the next briefing?
- **The interface is graded.** Concise first, on any device: what does the manager see in the first three lines? What is one click away? How do they get from a claim to the rows behind it?

Aspect	Details
Core capabilities	• A designed, documented tool set (retrieval / judgement / tracing / discovery / action) • Scheduled briefing plus all-day multi-turn Q&A with fresh retrieval • Standing watches and alerts • Feasibility estimates with stated assumptions • Confirmed, logged actions (messages, notes, reminders) • Full traceability from statement to source rows • **UX**: a clean, concise interface the manager would actually use — first three lines, one click to detail; any device (laptop is fine)
Example data	Provided: 120 orders, 90 days of production, 8 workshops — small and clean; extend it if you wish
Business impact	The planner's half hour every morning, plus every "let me check and call you back" during the day
Skills gained	Tool design for agents, retrieval over structured data, grounding and anti-hallucination, human-in-the-loop actions, interface design, evaluation design

Deliverable specific to this track: a **tool specification** (one table: name, input, output, purpose, non-goals, failure mode) plus an evaluation set of **at least 30 questions** with verified answers and your measured accuracy. At least 5 must be ambiguous, unanswerable, or designed to bait a hallucination, and at least 5 must be feasibility or action requests — not just lookups.

Track 2: The Quick-Response Subcontracting Desk

> ***In plain terms**: the factory works with **8 outside workshops**, each with a profile card — what it can make, how fast it works, how much work it is already holding, its defect rate, its price, and whether it is currently allowed to take work. Orders keep arriving, and for every order your system must **pick which workshop gets it**: can this shop make the product, can it take it now, and is it allowed to? Then it writes down why it chose that shop. To grade everyone fairly, a shared simulator replays the same 120 orders through your system and measures the results — and in one test run, a workshop suddenly closes for two weeks without warning, and a good system notices and sends work elsewhere.*

The setting: A reorder lands on Monday, due in ten days, and the factory is already full. The batch has to go to one of **eight outside workshops** — today, within the hour. That speed is the whole point of quick response: the factory that farms out a batch today beats the one that decides on Thursday.

How the decision is made now: somebody posts a message in a group chat.

> *"@Chen — order 042, 1,500 crewnecks. Send it to QuickStitch, need it back by the 19th."*

That message **is** the decision, and everything behind it lives in one dispatcher's head. Because for every workshop, three questions must be answered before a batch can go there:

1. **Can they make it?** Not every workshop is equipped for every product — some do knitted tops, some do accessories, some both (`makes` in `workshops.csv`).
2. **Can they take it now?** Capacity is per day, and every workshop is already holding a queue of work. A workshop that is "fast" but three days deep in backlog is not fast today.
3. **Are they allowed to take it?** One workshop is suspended after a quality audit; another is new and on trial, capped at small batches (`status` and `max_batch_pieces`).

Get any of the three wrong and the batch bounces back a day later — in a ten-day cycle, that is the margin gone.

Goal: Build the system that answers those three questions instantly, makes the allocation, and leaves behind what the group chat never did: **the decision and its reasoning, on record.**

The requests arrive as chat, not as rows — this is where the LLM earns its place

You are given the morning's inbox: `dispatch_requests.txt` , thirty allocation requests exactly as people type them. Some are complete. Some carry instructions ("*keep it away from BudgetWorks*"). Some are ambiguous ("*the TrendCart order*" — there are six) or missing a number, and the right move is a follow-up question. Some suggest a workshop that is not allowed to take the job, and the right move is a polite refusal with the reason and an alternative. Three are complete but **impossible** — no workshop can make the date, and one order is already overdue — and the right move is to say so with the numbers, give the earliest achievable date, and lay out the options. A few cannot be answered from this data at all. (`data_dictionary.md` gives the count of each kind, so you can check your labelling.)

Your system must take these messages as its input. An allocation policy with no natural-language surface — however good its decisions — does not meet this track's core capabilities, because the group chat *is* the interface being replaced. This is the LLM's real job in Track 2: extract the structured request, notice what is missing, ask, refuse, and explain — while the arithmetic stays in tools.

What the system must do

- **Take the requests as they come** — parse each chat message into a structured request; ask when it is ambiguous or incomplete; refuse (with the reason and an alternative) when the suggested workshop cannot take it; honour constraints and log who imposed them
- **Keep the workshop register** — capability, status, batch limits, current queue — and answer questions against it instantly: *"who can take 1,500 crewnecks this week?"*
- **Estimate before committing** — a promised-date estimate per candidate workshop (queue + processing + transport), so the choice is made on numbers, not reputation
- **Allocate every incoming order** with the reasoning attached: which workshops were eligible, what each would have cost in time and money, why this one won
- **Handle the awkward cases** — no eligible workshop free this week: split the batch? accept lateness? escalate to a human with options? An explicit answer here is worth more than a clever one. (Splits live in your chat layer and its record; the simulator sends one batch to one workshop.)
- **Produce an auditable record** — the structured artefact the group chat never produced

You may implement the allocation as a bidding round, a negotiation between agents, or a single well-tooled dispatcher agent — **the mechanism is your choice and is not what is graded.** A plain dispatcher with excellent tools beats an elaborate multi-agent conversation that produces a transcript instead of a decision.

The tools the agents need

As in Track 1, **specify the tools before you build them**: a capability-and-eligibility lookup, a current-queue check, a delivery-date estimator, a "what happens if we overload this workshop" probe. Each needs an input, an output, and a stated reason to exist.

The evaluation: the shared simulator

You are given a **shared simulator** (`harness/simulate.py` , ~350 lines, a quarter of it the explanatory header — read all of it) so that every team's results are comparable. It replays the 120 orders in date order, asks your system to choose a workshop for each, simulates the outcome from the profile cards (queues build up, defects happen, transport takes time), and reports turnaround, lateness, defect exposure, cost and load concentration. Choosing an ineligible workshop is an error, not a bad score — your system is expected to check first, like a real dispatcher.

What the simulator is, and is not. It is a *what-if replay*: suppose every one of the 120 orders had been sent outside on its `order_date` , starting 2026-01-01 — so it ignores the in-house columns (`status` , `current_stage` , `completed_date` , `days_late`). A workshop takes the whole job as one unit of work: no stages inside a workshop, the same capacity for every product, seven days a week, one batch to one workshop. That is deliberately simple — the data is not the challenge. And what it calls 120 times per run is your **allocation policy**: deterministic code, the same tool your chat agent calls. **Do not put an LLM call inside the allocator** — it buys nothing here and burns your API budget.

It ships with **three baseline allocators**, a few lines each (and even the dumbest baseline checks eligibility first):

- `random` — any eligible workshop. If you cannot beat this, something is wrong.
- `greedy_biggest` — always the highest-capacity workshop. Its queue detonates — that is the lesson about capacity.
- `cheapest` — always the lowest price. Great cost; look at everything else.

Beat all three on your primary objective, run at least one `--shock` scenario, and **report the cases where a baseline still wins**. "Beat" means on that objective's own metric, without giving up the basics:

Primary objective	The metric (from `report()`)	Guardrail

Minimise lateness	`% late` and `late days`	—
Minimise defects	`% def pcs` (defective batches weighted by size)	`% late` no worse than `random`
Fair allocation	`max share`	`% late` no worse than `random` — `random` is already near the floor on share, so matching its share while delivering on time is the win
Robust to failure	the *change* in `% late` and `late days` from the normal run to `--shock`, averaged over 10 seeds	`% late` no worse than `random` in the normal run

The shock scenario is the simulator's robustness test — a surprise in the middle of the run. With `--shock`, one workshop suddenly closes for two weeks (day 30 to 44 of the run, i.e. 31 Jan to 14 Feb 2026), like a shop failing an inspection mid-season. Your system is not told which one or when. Any batch sent to the closed workshop just sits at its door until it reopens — so a system that keeps sending work there delivers late, while a good one notices that the queue there has stopped moving and starts routing orders to the other workshops. The test separates a system that only works when everything goes to plan from one that adapts when something breaks. Which workshop closes depends on the seed, so one seed is one roll of the dice: **report `--shock` results averaged over ten seeds** (`python simulate.py --shock --seeds 10`; `average_over_seeds()` does the same for your allocator). A system that wins everywhere is usually being measured kindly; saying where and why a baseline wins is worth more than the win itself. If you think the simulator's assumptions are wrong, change them and say so — arguing with the harness, with evidence, is encouraged.

And the language surface is evaluated too. Run your system on all thirty requests in `dispatch_requests.txt` and report the results by behaviour: on complete requests, was the extraction right and the allocation sound? On ambiguous or incomplete ones, did it ask rather than guess? On ineligible suggestions, did it refuse with the correct reason and offer an alternative? On unanswerable ones, did it say so instead of inventing? Judge the decision and the explanation separately — **a right allocation with an invented justification is a failure**, because every number in the explanation must trace back to a tool output.

The objective is a configuration, not a hard-coded choice

There are four candidate objectives: **minimise lateness**, **minimise defects**, **keep allocation fair across workshops**, and **stay robust when a workshop fails**. No team has to optimise all four at once — but your system must take its objective as a **configuration**, not something baked into the allocation logic.

At **Sprint 1** you pick a **primary objective** to optimise hardest. You must also run the simulator under at least **one other objective and one hybrid setting** (a weighted mix of two or more — you choose the weights and justify them; put the metrics on a common scale first, since days, percentages and dollars do not add) and report how the allocations change. The same 120 orders allocated under three different objectives is where the interesting findings are, and at the end of the semester we compare them across teams. **Your findings are part of the deliverable, not just your code.**

Warnings

- **The LLM must not do the arithmetic.** Queue positions and delivery estimates come from tools; the agent decides over the results. An estimate hallucinated by the model is the Track 2 equivalent of a fabricated number in Track 1.
- **An allocation is a decision, not a conversation.** However the mechanism works inside, the output is a structured record a human could audit six weeks later.

- **Beating `random` is table stakes, not a result.** The interesting comparison is your full system against the best **simple rule-based allocator** you can write — no LLM, about the size of the three shipped baselines (for example "earliest estimated finish": queue + processing + transport, pick the smallest). The line count does not matter; the point is a rule a human could read in ten seconds. If that rule wins, say so and explain what your system adds (or honestly, that it adds nothing yet).

Optional extension: workshops that only do some stages

If you want a richer model, `simulate.py --stages` switches on a fourth eligibility question: *do they cover the stages this batch still needs?* It uses two extra files, `orders_stages.csv` (adds `dispatch_stage`) and `workshops_stages.csv` (adds `stages` — one workshop is a finishing house that only washes and packs, two have no knitting machines). A workshop can take a batch only if it covers every stage from the dispatch stage through PACKING; `eligible_workshops()` applies the rule for you, and nothing else about the simulator changes. **This is opt-in.** The default run is the common yardstick and its numbers are unchanged; if you use `--stages`, say so, and compare yourself with the `--stages` baselines.

Aspect	Details
Core capabilities	• Natural-language intake: chat requests parsed, clarified, refused or fulfilled • A queryable workshop register: capability, capacity, queue, eligibility • Per-candidate delivery estimates before committing • Allocation of every order with faithful reasoning attached, including the no-good-option cases • A documented tool set • Auditable structured output • Evaluation on the shared simulator against all three baselines (incl. a shock scenario) plus all 30 chat requests • **UX**: an interface the dispatcher would use under time pressure — candidates, estimates and the recommendation at a glance
Example data	Provided: 120 orders, 8 workshop profile cards, and 30 chat-style requests — small and clean; extend if you wish
Business impact	Direct: days of turnaround and points of defect rate, on the work that pays best
Skills gained	Agent and tool design, constraint checking, estimation under load, simulation, evaluation against baselines

---

Track 3: Research Paper Ideation and Writing — That Doesn't Read Like a Robot

Goal: Generate research paper ideas with a clear contribution, and produce a **structured draft** — Abstract, Introduction, Related Work, Methodology, Experiments, Conclusion.

This track has two halves, and most teams only do the first

Half 1 — the idea must be real. Grounded in actual published work. Search your target venues (NeurIPS, ICML, ACL, CVPR, EMNLP), understand what is already out there, and generate ideas that build on or diverge from it in a way you can defend. You do not need to run the full experiment; you need an idea a reviewer would take seriously.

Half 2 — the writing must not sound like an LLM wrote it

This is the distinguishing requirement of this track, and it is graded.

LLM prose has a smell. Everybody can detect it and nobody can quite define it. Some of the symptoms:

Symptom	Example
Empty scaffolding	"In today's rapidly evolving landscape of artificial intelligence…"
Hedge stacking	"This may potentially suggest that it could possibly be the case that…"
The rule of three, forever	"robust, scalable, and efficient" — every single sentence
Promotional adjectives	"a groundbreaking, transformative paradigm shift"
Symmetrical paragraphs	Every paragraph exactly four sentences, every section exactly three paragraphs
Contentless transitions	"Furthermore, it is important to note that…"
Fake balance	"While X has advantages, it also has limitations" (naming neither)

Real academic writing has an argument, admits uncertainty in specific places, has sentences of wildly varying length, and occasionally says something blunt.

What you must deliver for Half 2 (the default evaluation — you may propose your own of equivalent rigour):

1. **A detector** — measure how "AI-sounding" a draft is. Build it however you like: stylometric features (sentence-length variance, hedge density, adjective ratio, n-gram repetition), a classifier trained on real papers vs LLM output, an LLM-as-judge rubric, or a combination. Justify your choice.
2. **A rewriting loop** — the generator writes, the detector scores, the system revises. Show the score improving across iterations.
3. **A human evaluation** — recruit at least 3 people (classmates count). Show them mixed real and generated paragraphs and ask them to identify which is which. **Report the accuracy honestly.** Fooling humans 50% of the time is a perfect score; 90% detection means you have work left. A team that reports 85% detection with a sharp analysis of *why* will score better than a team claiming 50% with no evidence.
4. **Before/after examples** in your report — same content, dry version and revised version, side by side.

Warning about the obvious shortcut: telling the model "write in a human way, avoid AI clichés" produces a slightly different flavour of AI writing. It usually helps a little and then plateaus. Measure it, show the plateau, and go further.

Aspect	Details
Core capabilities	• Ground ideas in real literature • Produce complete structured drafts • **Measure** and **reduce** AI-sounding prose • Validate with human readers • **UX**: a working interface for the write–score–revise loop — drafts, detector scores and before/after diffs visible, not buried in a notebook
Example data	Semantic Scholar, arXiv, ACL Anthology, OpenReview
Business impact	Faster literature-grounded drafting; output that survives a reader
Skills gained	Agentic workflows, evaluation design, stylometry, human evaluation

References: AI-Scientist, AI-Scientist-v2, Wikipedia's *Signs of AI writing*

Track 4: Student Productivity Agent

Goal: An agent that plugs into the tools you already use — Outlook, Teams, your calendar — and takes the administrative load off a student's week.

> ***Canvas / LMS is out of scope.*** *NUS IT has confirmed that Canvas API access depends on the user's role, requires approval from system administrators and sits outside their normal process because Canvas is a critical system.* ***Do not generate a Canvas access token and do not call*** `canvas.nus.edu.sg` ***for this project.***

Why this track exists: You are the user. You know exactly where the pain is, which means you have no excuse for building something that misses it.

Core capabilities

1. Inbox triage and drafting

- Read the inbox, separate "needs a reply", "needs an action", "ignore"
- Draft replies **in your voice** — trained or few-shot prompted on your own sent mail
- Handle the awkward ones properly: asking a professor for an extension, chasing a groupmate who has gone quiet, replying to a recruiter

2. Deadline extraction

- Deadlines arrive as prose scattered across email, Teams messages and PDF course outlines — "*the report is due the Friday after reading week*"
- Extract them, resolve relative dates against the academic calendar, deduplicate the same deadline mentioned in four places, and put them in one list
- **Flag conflicts**: three deadlines and a midterm in the same 48 hours

3. Study planning

- Given deadlines, estimated effort, and the gaps in your calendar, produce a realistic plan
- "Realistic" is the hard part: an agent that schedules 14 hours of work on Wednesday is useless. It needs to respect how people actually work — limited deep-work hours per day, buffer before deadlines, no 2 a.m. sessions
- Re-plan when things slip, because things slip

4. Meeting and group-work support

- Summarise a Teams thread into decisions and action items
- Draft the agenda; after the meeting, draft the follow-up

Integration guidance

Microsoft Graph API covers Outlook, Teams and Calendar; Google Workspace APIs are the equivalent if you live on Gmail. **There is no LMS integration in this track** — see the note above. **Start read-only.** Get permission scopes right, and be deliberate about what a mistake costs.

The thing that will make or break this project

An agent with write access to your email can embarrass you. Design for that from the start:

- What can it do without asking? (draft a reply — safe)
- What must it always ask about? (send anything, decline a meeting, email staff)
- How does the user see what it is about to do before it does it?
- What happens when it misreads a deadline and cancels your Friday?

A team that ships a clean confirm-before-send flow and can articulate the failure modes will outscore a team with twice the features. **Discuss privacy explicitly**: whose mail is being sent to which model provider, and what did you do about it?

Aspect	Details
Core capabilities	• Inbox triage and voice-matched drafting • Deadline extraction from unstructured text across sources • Realistic, re-plannable study schedules • Thread summarisation and follow-ups • **UX**: a daily-driver interface with a clean confirm-before-send flow — you should *want* to use it yourself
Example data	Microsoft Graph API, Google Workspace API, your own mailbox
Business impact	Hours per week per student; directly transferable to any knowledge worker
Skills gained	OAuth and API integration, temporal reasoning, planning under constraints, human-in-the-loop safety design

Evaluation (example — you may propose your own of equivalent rigour): **time saved** — how many hours per week, and on what (inbox triage, deadline tracking, planning) — measured against doing the same tasks manually.

Alternatives of equal standing, depending on where your team focuses: **deadline-extraction accuracy** on at least 50 real messages, with false positives and misses reported separately — a missed deadline and an invented one are different failures, and both matter; **triage accuracy** on ~100 hand-labelled real emails (needs-reply / needs-action / ignore, with a confusion matrix); a **blind voice study** — can readers tell your real sent mail from the agent's drafts?; a **planning constraint check** — across many generated plans, count double-bookings, overloaded days and work scheduled after its deadline; or a **safety red-team** — a set of risky requests verifying nothing escapes the confirm-before-send flow.

Track 5: Propose Your Own

Got a unique idea? Submit a proposal including:

- Problem statement and target users
- Core capabilities your system should demonstrate
- Technical approach and feasibility
- Expected business impact
- **How you will evaluate it** — the most common reason proposals are rejected

The difficulty must match Tracks 1–4. Use them as the bar: a designed and documented tool set, an interface a real user would use (UX is graded here too), and an evaluation with baselines you have to beat. A proposal that is a thin wrapper around one prompt, or that skips the evaluation, will be sent back for another round.

Approval required: email your instructor by Week 3.
